

爱予慧AI智能体管理服务后端 V1.0

设计说明书

著作权人：上海爱予慧科技有限责任公司

版本号：V1.0

文档版本：2026-09-01

一、引言

1.1 编写目的

本设计说明书旨在全面阐述"爱予慧AI智能体管理服务后端 V1.0"（以下简称"本系统"）的总体架构、模块划分、接口设计、数据存储与关键技术，为软件著作权登记提供技术文档依据，并为后续维护与二次开发提供设计参照。本系统是上海爱予慧科技有限责任公司 AI 语音陪伴硬件产品线的管理后台 RESTful 服务，为智控台 Web 端（manager-web）与移动管理端（manager-mobile）提供 API，同时作为 Python 语音核心服务（xiaozhi-server）的配置数据来源。本系统由上海爱予慧基于开源项目 xiaozhi-esp32-server（MIT 协议，著作权人 xinnan-tech）二次开发形成，在设备管理、智能体配置、NFC 写卡与领取、声音克隆、远程配置下发、故事引擎、支付订阅等模块新增了大量独创代码。

1.2 项目背景

AI 语音陪伴硬件产品线包含 ESP32 语音终端、Python 语音核心服务、智控台管理平台、移动管理端、蛋宝宝小程序与数字人演示端多个组成部分。其中智控台与移动端需要一个统一的后端服务承载设备注册与绑定、智能体配置管理、OTA 固件升级、NFC 写卡批次与领取、声音克隆任务、知识库管理、支付订阅、故事引擎配置等业务。本系统即为此而生，以 Java Spring Boot 提供高并发、强类型、易维护的 RESTful API 服务，对外暴露统一上下文路径 /xiaozhi，运行于 8002 端口。

本系统同时承担配置中枢职责：运营人员在智控台修改智能体参数（ASR/TTS/LLM 选型、提示词、记忆策略等），经本系统持久化至 OceanBase 后，Python 语音核心服务通过远程配置拉取接口获取最新配置并热重载，无需重启即生效。该机制使产品线的 AI 能力可在运行时动态调整。

1.3 术语定义

术语	说明
REST	Representational State Transfer，表述性状态转移，HTTP 资源化接口风格
Spring Boot	Java 企业级应用快速开发框架，约定优于配置
MyBatis-Plus	MyBatis 增强工具，提供 CRUD 通用方法与条件构造器
Shiro	Apache Shiro，Java 轻量级安全框架，负责认证鉴权与会话管理
Liquibase	数据库变更追踪工具，以 changelog 管理版本化迁移
Knife4j	SpringDoc OpenAPI 的增强 UI，生成接口文档与调试页面

Druid	阿里巴巴数据库连接池，提供监控与 SQL 解析
OTA	Over The Air，空中固件升级，通过 HTTP 下发设备固件
NFC	近场通信，本系统中指 NFC 卡片写卡与领取业务
智能体	Agent，一台语音设备的 AI 配置单元，绑定 ASR/TTS/LLM/提示词等
OceanBase	蚂蚁集团分布式关系数据库，兼容 MySQL 协议
智控台	manager-web，Vue.js 管理控制台前端，本系统的对接方之一
语音核心服务	xiaozhi-server，Python asyncio 语音 AI 服务，本系统的下游配置消费方

1.4 参考标准

- 《计算机软件著作权登记办法》及中国版权保护中心 R11 登记指南
- GB/T 8567-2006 《计算机软件文档编制规范》
- MIT 开源许可证（上游 xiaozhi-esp32-server 许可证明）
- Spring Boot 3.4.3 官方文档
- RFC 7519 JSON Web Token (JWT)
- OpenAPI 3.0 规范

1.5 文档范围

本说明书覆盖本系统的总体架构、鉴权安全、设备管理、智能体管理、远程配置下发、NFC 写卡领取、宠物与陪伴上下文、声音克隆、知识库、支付订阅、故事引擎、系统管理、数据存储、接口文档、配置部署、关键技术与安全设计共二十章。源代码鉴别材料另册提交，与本说明书共同构成本系统完整的著作权登记文档。

二、系统概述

2.1 软件用途

本系统为爱予慧 AI 语音陪伴硬件产品线提供统一的管理后端 RESTful API 服务。前端管理控制台（manager-web）与移动管理端（manager-mobile）通过 HTTP 调用本系统的接口，完成设备注册与绑定、智能体配置、OTA 固件升级、NFC 写卡批次与领取管理、声音克隆任务、知识库管理、支付与订阅、故事引擎配置、系统字典与参数维护等业务操作。同时本系统作为配置中枢，将运营人

员调整的智能体参数与模型选型持久化后供 Python 语音核心服务远程拉取，实现全产品线 AI 能力的运行时动态下发。本系统是产品线的管理中枢与配置数据源。

2.2 运行环境

2.2.1 服务端环境

- 操作系统：Linux（Ubuntu 22.04 及以上）
- 运行时：Java 21（LTS）
- 框架：Spring Boot 3.4.3
- 构建工具：Maven 3.8+
- 依赖服务：OceanBase（MySQL 8.0+ 兼容，业务数据与配置存储）、Redis 5.0+（会话与缓存）
- 网络：对外开放 HTTP 端口 8002，上下文路径 /xiaozhi
- 容器：提供 Dockerfile 与 docker-compose 一键部署

2.2.2 客户端环境

- 智控台：manager-web（Vue.js 2），运行于端口 8001（开发）或经 Nginx 代理
- 移动管理端：manager-mobile（Uni-app），输出 H5、微信小程序、iOS、Android 多端
- Python 语音核心服务：xiaozhi-server，通过 HTTP 拉取配置

2.3 技术栈

层次	技术选型	用途
基础框架	Spring Boot 3.4.3 / Java 21	自动装配、内嵌容器、依赖注入
数据访问	MyBatis-Plus 3.5.5	ORM、通用 CRUD、条件构造器、分页
连接池	Druid	数据库连接池、SQL 监控
认证鉴权	Apache Shiro 2.0.2（Jakarta）	过滤器链、会话、权限注解
缓存	Spring Data Redis	会话存储、业务缓存
数据库迁移	Liquibase	版本化 changelog 迁移
接口文档	Knife4j 4.6.0 / SpringDoc	OpenAPI 文档与调试 UI
对象存储	OSS 抽象层	固件包、声音样本、头像上传
日志	Logback	结构化运行日志

参数校验	Hibernate Validator	Bean Validation 注解校验
------	---------------------	----------------------

2.4 主要功能

- 1. 设备管理：设备注册、绑定/解绑智能体、手动添加、设备工具列表与调用、通讯录与权限。
- 2. 智能体管理：智能体配置 CRUD、快照、声纹、MCP 接入点、模板、聊天历史。
- 3. OTA 固件升级：固件元数据与二进制包下发，MD5 校验，设备版本查询。
- 4. 远程配置下发：智能体参数持久化，供 Python 服务端拉取热重载。
- 5. NFC 写卡与领取：产品类型、批次、资产、写卡任务、领取、Scheme、操作日志全生命周期管理。
- 6. 宠物与陪伴上下文：宠物档案、上下文、记忆、聊天、故事管理。
- 7. 声音克隆：声音样本上传、克隆任务提交、声音资源管理。
- 8. 知识库：知识库与知识文件管理，供检索增强。
- 9. 支付与订阅：订单、支付通知、订阅周期管理。
- 10. 故事引擎：故事编排与上下文配置。
- 11. 系统管理：管理员、字典类型与数据、参数、操作日志、服务端管理。
- 12. 鉴权安全：Shiro 过滤器链、Token 会话、权限注解、XSS 过滤、全局异常。

2.5 用户角色

角色	说明
管理员	通过智控台或移动端登录本系统，管理设备、智能体、业务配置的运营人员
智控台前端	manager-web，本系统的 Web API 消费方
移动管理端	manager-mobile，本系统的移动 API 消费方
Python 服务端	xiaozhi-server，远程配置拉取方，本系统的下游消费方
设备	ESP32 终端，经本系统注册与绑定后接入语音服务

三、系统架构设计

3.1 总体架构

本系统采用经典分层架构，自上而下分为接入层、控制层、服务层、持久化层四层。接入层即 Controller 层，以 @RestController 暴露 RESTful 接口，统一上下文路径 /xiaozihi；控制层由 Shiro 过滤器链、全局 Handler 与 Aspect 切面组成，负责认证鉴权、异常处理、参数解析、操作日志与幂等切面；服务层为各业务模块的 Service，承载全部业务逻辑；持久化层以 MyBatis-Plus DAO 访问 OceanBase，辅以 Redis 缓存与会话存储，并由 Liquibase 管理数据库版本迁移。层间依赖单向向下，上层不感知下层具体实现，通过接口与依赖注入解耦。

本系统对外承接两类前端（manager-web、manager-mobile），对下通过配置下发接口与 Python 语音核心服务对接，是产品线管理数据的中枢。接口文档由 Knife4j 自动生成，访问地址 /xiaozihi/doc.html。

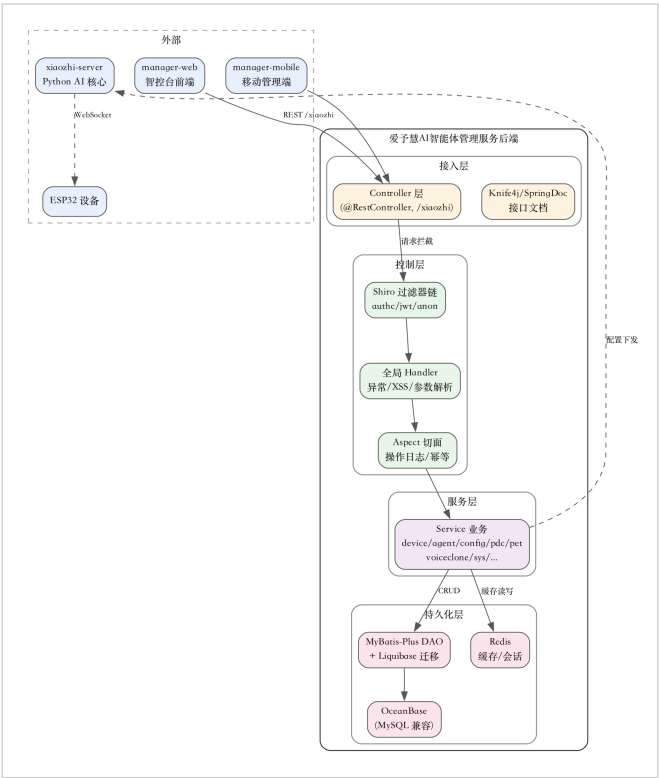


图 1 系统分层架构图（接入/控制/服务/持久化四层 + 外部依赖）

3.2 分层设计

层次	模块	职责
接入层	各模块 controller 包	HTTP 路由、参数绑定、响应封装

控制层	common/interceptor、common/handler、common/aspect	Shiro 鉴权、全局异常、XSS、操作日志、幂等
服务层	各模块 service 包	业务逻辑编排、事务边界、领域校验
持久化层	各模块 dao 包 + common/redis	MyBatis-Plus CRUD、Redis 缓存

3.3 包结构

```

src/main/java/xiaozhi/
├── AdminApplication.java           # Spring Boot 入口
├── common/                         # 基础设施
│   ├── config/                   # ShiroConfig/MybatisConfig/RedisConfig/
│   │   └── Knife4jConfig
│   ├── exception/                # 全局异常与业务异常码
│   ├── handler/                  # 全局响应封装与异常处理
│   ├── interceptor/              # 鉴权与日志拦截器
│   ├── aspect/                   # 操作日志/幂等切面
│   ├── constant/                 # 系统常量
│   ├── utils/                    # 通用工具
│   ├── annotation/               # 自定义注解
│   ├── convert/                  # 类型转换器
│   ├── page/                     # 分页封装
│   ├── validator/                # 自定义校验器
│   ├── xss/                      # XSS 过滤
│   ├── oss/                      # 对象存储抽象
│   ├── upload/                   # 上传服务
│   ├── redis/                    # Redis 工具与配置
│   ├── service/                  # 通用服务
│   ├── dao/                      # 通用 DAO
│   ├── entity/                   # 基础实体
│   └── user/                      # 用户上下文
└── modules/                      # 业务模块
    ├── device/    agent/    config/    security/
    ├── pdc/       pet/      voiceclone/ sys/
    ├── storyengine/ knowledge/ payment/
    └── subscription/ model/    timbre/

```

```
├─ wechat/   invite/   companion/
├─ feedback/ item/     correctword/
├─ email/    sms/      llm/
```

3.4 模块划分

业务模块位于 `modules/` 下，每个模块遵循 `controller / service / dao / entity / dto` 子包结构，职责单一、高内聚低耦合。`device` 模块管理设备全生命周期，`agent` 模块管理智能体配置，`config` 模块管理远程配置下发，`pdc` 模块（含 `nfc` 子模块）管理 NFC 写卡与领取全链路，`pet` 模块管理宠物档案与陪伴上下文，`voiceclone` 管理声音克隆，`sys` 管理系统字典与参数，`storyengine` 管理故事引擎，`knowledge` 管理知识库，`payment` 与 `subscription` 管理支付订阅。各模块通过 `Service` 接口对外暴露能力，`Controller` 仅做协议转换，不承载业务逻辑。

3.5 应用配置

应用入口 `AdminApplication` 以 `@SpringBootApplication` 注解启用自动装配与组件扫描。核心配置项集中于 `application.yml`，主要配置段示例如下：

```
server:
  port: 8002
  servlet:
    context-path: /xiaozihi
  tomcat:
    uri-encoding: UTF-8
    threads:
      max: 200

spring:
  datasource:
    driver-class-name: com.mysql.cj.jdbc.Driver
    url: jdbc:mysql://seekdb:2883/xiaozihi?...
    druid:
      initial-size: 5
      max-active: 20
  redis:
    host: seekdb
    port: 6379
```



```
mybatis-plus:
  mapper-locations: classpath*:mapper/**/*.xml

knife4j:
  enable: true
  setting:
    language: zh-CN
```

敏感字段（数据库密码、Redis 密码、Token 密钥）经环境变量或 application-{profile}.yaml 注入，不固化于代码仓库。

3.6 全局响应与异常处理

common/handler/Result 封装统一响应，结构含 code（业务码）、msg（消息）、data（数据载荷）。common/exception/BusinessException 携带错误码与消息，由 @RestControllerAdvice 全局捕获并脱敏返回。错误码分区：0 成功、1xx 客户端参数、4xx 鉴权、5xx 服务端，便于快速定位。该机制保证接口响应结构一致，前端按 code 分支处理即可。

四、鉴权与安全设计

4.1 Shiro 过滤器链

本系统采用 Apache Shiro 2.0.2 (Jakarta classifier) 作为安全框架。ShiroConfig 注册 ShiroFilterFactoryBean，定义过滤器链（filterChainDefinitionMap），按路径模式匹配将请求分发至 anon（匿名放行）、authc（需认证）、jwt（JWT Token 校验）等过滤器。登录、接口文档、OTA 固件查询等端点归入 anon 白名单；其余业务接口归入 jwt 或 authc，要求携带有效 Token。过滤器链顺序由具体性优先原则保证最具体的路径模式先匹配。

```
@Bean
public ShiroFilterFactoryBean shiroFilter(SecurityManager sm) {
    ShiroFilterFactoryBean bean = new ShiroFilterFactoryBean();
    bean.setSecurityManager(sm);
    Map<String, Filter> filters = new HashMap<>();
    filters.put("jwt", new JwtFilter()); // 自定义 JWT 过滤器
    bean.setFilters(filters);
    Map<String, String> chain = new LinkedHashMap<>();
    chain.put("/admin/login", "anon"); // 登录放行
```

```

chain.put("/doc.html", "anon");           // 接口文档
chain.put("/swagger**", "anon");
chain.put("/ota/**", "anon");             // OTA 查询放行
chain.put("/**", "jwt");                  // 其余需鉴权
bean.setFilterChainDefinitionMap(chain);
return bean;
}

```

LinkedHashMap 保证按插入顺序匹配，最具体的 anon 规则先于兜底的 /** jwt 规则生效。

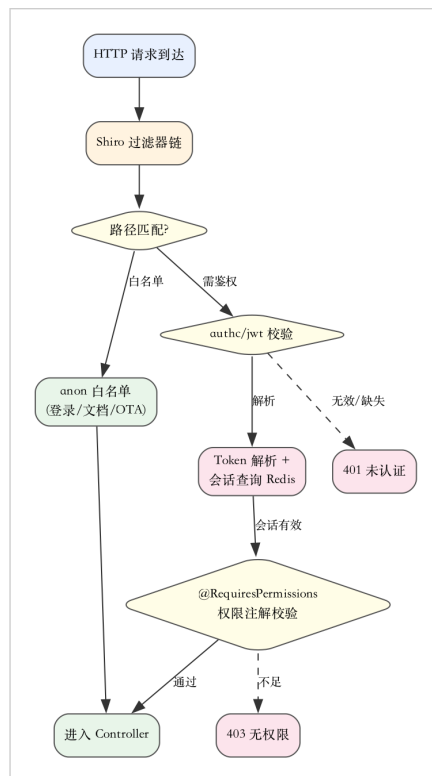


图 2 鉴权流程图（Shiro 过滤器链 + Token 校验 + 权限注解）

4.2 Token 会话

登录成功后由 LoginController 签发 JWT Token，载荷含管理员标识与过期时间，签名密钥配置于 application.yml。后续请求在 Header 携带 Token，Shiro 的 jwt 过滤器解析校验，通过后将会话写入 Redis 并绑定至当前线程上下文（common/user）。会话有效期由 Redis TTL 控制，支持滑动续期。登出接口清除 Redis 会话并使 Token 失效。

4.3 权限注解

细粒度权限通过 Shiro 的 @RequiresPermissions 注解在 Controller 方法或 Service 上声明所需权限标识。权限数据由 sys 模块的字典与角色关系维护，管理员登录后其权限集合载入会话。鉴权失败抛

出未授权异常，由全局异常处理器统一包装为标准错误响应（HTTP 403）。该设计将权限规则与业务代码解耦，运营端调整权限无需改代码。

4.4 XSS 与参数校验

common/xss 提供 XSS 过滤器，对请求参数中的危险脚本标签做转义与清除。参数校验采用 Hibernate Validator（Bean Validation），在 DTO 上以 @NotBlank/@Length/@Pattern 等注解声明约束，common/validator 提供自定义校验注解（如手机号、字典键）。校验失败由全局异常处理器捕获并返回字段级错误信息，避免非法数据进入业务层。

4.5 全局异常与响应封装

common/handler 定义统一响应封装 Result<T>（含 code/msg/data）与全局异常处理器 @RestControllerAdvice。业务异常 BusinessException 携带错误码与消息，系统异常被兜底捕获并脱敏后返回统一错误结构。该机制保证接口响应结构一致，便于前端统一处理与日志追溯。

五、设备管理模块设计

5.1 模块职责

device 模块（modules/device）管理 ESP32 终端设备的全生命周期，包括设备注册、绑定至智能体、解绑、手动添加、设备工具列表与调用、通讯录与权限。DeviceController 暴露 RESTful 接口，DeviceService 承载业务逻辑，DeviceDao 经 MyBatis-Plus 访问设备表。设备是语音服务的接入主体，必须先注册并绑定智能体后方可接入 Python 语音核心服务。

5.2 核心接口

方法	路径	说明
POST	/device/register	设备注册，录入 MAC 与设备码
POST	/device/bind/{agentId}/{deviceCode}	设备绑定至指定智能体
POST	/device/bind/{agentId}	已注册设备绑定智能体
GET	/device/bind/{agentId}	查询智能体下已绑定设备
POST	/device/unbind	解绑设备
PUT	/device/update/{id}	更新设备信息
POST	/device/manual-add	手动添加设备

POST	/device/tools/list/{deviceId}	设备工具列表
POST	/device/tools/call/{deviceId}	调用设备工具
GET	/device/address-book/{macAddress}	查询通讯录
PUT	/device/address-book/alias	设置通讯录别名
PUT	/device/address-book/permission	设置通讯录权限

5.3 设备与智能体绑定关系

设备（Device）与智能体（Agent）为多对一关系：一台设备同一时刻绑定一个智能体，一个智能体可被多台设备绑定。绑定关系存于设备表的 agent_id 外键。绑定校验由 DeviceService.bindDevice 完成，确认设备已注册、智能体存在且属同一租户后写入绑定关系，并通知 Python 语音核心服务刷新该设备的配置缓存。解绑时清理绑定关系并同步通知。

5.4 OTA 固件升级

OTAController（modules/device/controller/OTAController）提供固件升级能力。管理员上传固件二进制包与版本元数据（版本号、适用型号、MD5、发布说明），固件包存于对象存储，元数据落库。设备查询升级接口返回最新适用固件元数据与下载地址，设备下载二进制包并校验 MD5 后烧录。该机制支持产品线固件迭代下发，无需设备返厂。

5.5 设备实体

DeviceEntity 主要字段如下，其余模块实体结构类似，均以 id 主键、creator/updater 审计字段、create_date/update_date 时间戳为通用约定。

字段	类型	说明
id	String	主键，雪花或 UUID
device_code	String	设备码，唯一
mac_address	String	MAC 地址
agent_id	String	绑定智能体 ID（外键）
model	String	设备型号
state	Integer	状态（未绑定/已绑定/已禁用）
remark	String	备注

creator / updater	Long	创建/更新人
create_date / update_date	Date	创建/更新时间

5.6 设备绑定流程

设备绑定由 DeviceService.bindDevice 编排：校验设备已注册且未绑定其他智能体 → 校验智能体存在且同租户 → 写入 agent_id 外键 → 发布配置变更通知至 Python 服务端。绑定接口提供按设备码绑定与按智能体批量绑定两种方式。解绑接口清理 agent_id 并同步通知，保证语音服务端配置与关系库一致。

六、智能体管理模块设计

6.1 模块职责

agent 模块（modules/agent，89 个类）管理智能体（Agent）的配置全生命周期，是本系统最核心的业务模块之一。智能体是一台语音设备的 AI 配置单元，绑定 ASR/TTS/LLM 服务商选型、系统提示词、记忆策略、工具集合、声纹、MCP 接入点等。AgentController 提供 CRUD，AgentSnapshotController 提供配置快照回溯，AgentVoicePrintController 管理声纹，AgentMcpAccessPointController 管理 MCP 接入点，AgentTemplateController 提供模板快速创建，AgentChatHistoryController 管理聊天历史。

6.2 核心实体

实体	说明
Agent	智能体主配置，含名称、模型选型、提示词、记忆开关
AgentSnapshot	配置快照，支持历史版本回溯
AgentVoicePrint	声纹样本与说话人标识
AgentMcpAccessPoint	MCP 工具接入点配置
AgentTemplate	智能体模板，预设模型与提示词
AgentChatHistory	聊天历史记录

6.3 配置快照与回溯

智能体配置变更前由 AgentSnapshotController 自动生成快照，记录变更前的完整配置 JSON 与变更人、时间、原因。快照支持列表查询与回溯恢复：运营人员误操作后可选择历史快照一键回滚，避免配置错误导致线上语音服务异常。快照数据存于 OceanBase，按 agent_id 索引，保留策略可配置。

6.4 声纹与多用户

AgentVoicePrint 管理智能体关联的声纹样本。声纹用于说话人识别，使同一台设备能区分不同用户并加载各自记忆与画像。声纹样本经录音上传后入库，与智能体绑定。该模块与 Python 服务端的声纹 Provider 联动：本系统管理声纹元数据，Python 端执行声纹比对。

6.5 MCP 接入点

AgentMcpAccessPoint 管理智能体的 MCP (Model Context Protocol) 工具接入点。运营人员为智能体配置可调用的 MCP 工具来源（服务端工具、IoT 控制、外部 MCP 服务），配置下发后 Python 服务端的 UnifiedToolHandler 据此调度工具。该设计使智能体的工具能力可动态装配，无需改代码。

6.6 智能体配置结构

智能体配置以 JSON 形式聚合各能力选型，下发至 Python 服务端后由 Provider 工厂实例化。配置结构示例：

```
{
  "agent_id": "agt-001",
  "name": "客厅陪伴助手",
  "llm": {
    "selected_module": "OpenAI",
    "model": "gpt-4o-mini",
    "temperature": 0.7,
    "system_prompt": "你是用户的朋友..."
  },
  "asr": { "selected_module": "aliyun" },
  "tts": { "selected_module": "edge", "voice": "zh-CN-XiaoxiaoNeural" },
  "vad": { "selected_module": "silero" },
  "intent": { "selected_module": "keyword" },
  "memory": { "selected_module": "powermem", "enable": true },
  "tools": ["get_weather", "iot_control"],
```

```
"voiceprint": { "enable": true, "multi_user": true }
}
```

配置变更经 ConfigService 持久化后，Python 服务端拉取热重载，selected_module 切换即换服务商，无需重启。

七、远程配置下发设计

7.1 配置中枢职责

本系统是产品线的配置中枢。运营人员在智控台修改智能体参数（ASR/TTS/LLM 选型、提示词、记忆策略、模型温度等），经 ConfigController 接收后由 ConfigService 持久化至 OceanBase 配置表。Python 语音核心服务（xiaozi-server）启动时及运行中通过 GET /xiaozi/config 拉取最新配置，与本地 config.yaml 和 .config.yaml 三层递归合并后热重载 Provider，无需重启即生效。该机制使产品线 AI 能力可在运行时动态调整。

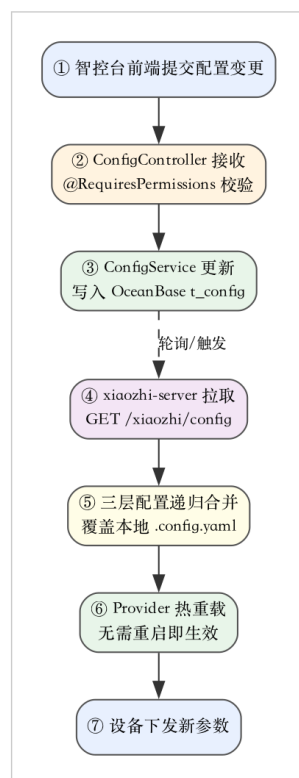


图 3 远程配置下发时序图（智控台→管理服务后端→Python 服务端→热生效）

7.2 配置数据模型

配置以智能体为粒度存储，每个智能体一条配置记录，含 ASR/TTS/LLM/VAD/Intent/Memory 各能力的 selected_module 选型与参数 JSON。配置变更采用乐观锁（version 字段）防并发覆盖。配置表结构由 Liquibase changelog 版本化管理，支持字段演进。

7.3 拉取接口

Python 服务端通过 GET /xiaozhi/config（携带设备标识或智能体标识）拉取配置。接口返回该智能体的完整配置 JSON。Python 端按固定间隔轮询或在智控台触发变更后主动通知拉取。接口归入鉴权过滤，需携带有效 Token 或服务间密钥。

7.4 三层配置合并

Python 服务端配置加载采用三层递归合并：config.yaml（默认）→ .config.yaml（本地密钥覆盖）→ 远程本系统配置（运行时下发）。后层覆盖前层，保证本地默认值、密钥与远程动态配置各司其职。合并后 Provider 工厂按 selected_module 重新实例化能力，旧实例释放。该设计实现了配置的热更新与多实例一致性。

八、NFC 写卡与领取模块设计

8.1 模块职责

pdn 模块下的 nfc 子模块（modules/pdn/nfc）管理 NFC 卡片写卡与领取的全生命周期，是本系统业务最复杂的模块之一（112 个类）。该模块支撑蛋宝宝小程序的 NFC 卡片领取业务：运营人员在智控台创建产品类型与批次，写入卡片资产，下发写卡任务，用户在小程序领取卡片后激活。模块包含产品类型、批次、资产、写卡任务、领取、Scheme、操作日志七个控制器，覆盖完整业务链路。

8.2 核心控制器

控制器	职责
PdcNfcProductTypeAdminController	产品类型管理（卡面、属性模板）
PdcNfcBatchAdminController	批次管理（创建、状态流转）
PdcNfcAssetAdminController	卡片资产管理（入库、绑定）
PdcNfcWriteJobAdminController	写卡任务管理（下发、回调、重试）
PdcNfcClaimController	用户领取（小程序侧调用）
PdcNfcSchemeAdminController	Scheme 配置（deeplink 协议）
PdcNfcOperationLogAdminController	操作日志审计

8.3 批次状态机

批次（Batch）是写卡业务的组织单元，具有状态机：创建（草稿）→ 待写卡 → 写卡中 → 写卡完成 → 已发放 → 已关闭。状态流转由 BatchService 显式驱动，每次流转记录操作日志。写卡失败可回退至待写卡重试。该状态机保证写卡业务的可追溯与防重复。

8.4 写卡任务与领取

写卡任务（WriteJob）描述一张卡片的写入指令，含目标资产、写入数据、状态（待执行/成功/失败）。任务下发至写卡终端执行，结果回调更新状态。领取（Claim）由用户在小程序扫描 NFC 卡片触发，ClaimController 校验卡片有效性后将其绑定至用户账户并激活。Scheme 配置定义 deeplink 协议，使小程序能通过 NFC 感应唤起领取流程。

8.5 操作日志审计

OperationLog 记录批次创建、状态流转、写卡任务下发与回调、领取等全部关键操作的执行人、时间、前后值。日志不可篡改，用于运营追溯与异常排查。该设计满足业务合规审计要求。

8.6 批次实体与状态流转

PdcNfcBatchEntity 是写卡业务的组织单元，承载状态机。主要字段与状态流转如下：

字段	说明
batch_no	批次号，唯一
product_type_id	产品类型外键
total / written / claimed	计划/已写/已领数量
state	状态机当前态
scheme_id	关联 Scheme 配置
operator	当前操作人

状态流转图：草稿(DRAFT) → 待写卡(PENDING) → 写卡中(WRITING) → 写卡完成(WRITTEN) → 已发放(ISSUED) → 已关闭(CLOSED)。写卡失败可回退至 PENDING 重试。每次流转由 BatchService.changeState 显式驱动并落 OperationLog，禁止跳跃流转。

8.7 写卡任务回调

写卡终端执行任务后回调 PdcNfcWriteJobController.updateResult，携带任务 ID、状态（成功/失败）、写卡数据。回调采用幂等设计：同一任务 ID 重复回调仅首次生效，状态机保证不重复发放。回调失败由写卡终端按指数退避重试，超过阈值标记异常待人工介入。

九、宠物与陪伴上下文模块设计

9.1 模块职责

pet 模块（modules/pet，52 个类）管理蛋宝宝小程序后端的宠物档案、陪伴上下文、记忆、聊天与故事数据。该模块为小程序的 AI 宠物孵化与陪伴玩法提供数据支撑，是蛋宝宝产品的后端业务核心。包含 PetController、ProfileController、MemoryController、ChatController、PetStoryController、PetContextController 等控制器。

9.2 实体关系

实体	说明
Pet	宠物档案，含品种、阶段、属性
Profile	用户档案，与宠物关联
Memory	宠物记忆条目
Chat	聊天记录
PetStory	宠物故事节点
PetContext	陪伴上下文（当前阶段、心情、状态）

9.3 陪伴上下文

PetContext 维护宠物的当前陪伴状态：孵化阶段、心情值、饱腹度、亲密度等动态属性。上下文由 ChatController 在每次对话后更新，驱动宠物成长与剧情推进。上下文缓存于 Redis 以支撑高频读写，定期落库。该设计将宠物养成玩法的状态与对话记忆解耦，便于独立演进。

9.4 记忆与聊天

MemoryController 管理宠物记忆条目，记忆条目可由 LLM 自动提取或运营手动维护，供下次对话增强上下文。ChatController 记录用户与宠物的聊天历史，支持分页查询与回放。聊天数据与 Python 服务端的 PowerMem 记忆联动：本系统管理记忆元数据，Python 端执行向量检索与召回。

9.5 宠物实体与成长阶段

字段	说明
pet_id	宠物唯一标识
user_id	所属用户
stage	成长阶段（蛋/幼体/成体）
mood	心情值（0-100）
fullness	饱腹度（0-100）
intimacy	亲密度（0-100）
story_node	当前故事节点

成长阶段由陪伴上下文与对话频次驱动：幼体阶段需维持饱腹度与心情，达标后进化为成体。阶段进化触发 StoryEngine 推进剧情分支。该设计将宠物养成玩法的状态机显式化，便于运营调参与剧情编排。

9.6 聊天记录结构

```
// 聊天记录
{
  "chat_id": "ch-001",
  "pet_id": "pet-001",
  "user_id": "u-001",
  "role": "user",           // user / assistant
  "content": "今天想出去晒太阳",
  "memory_extracted": ["喜欢晒太阳", "天气晴"],
  "create_date": "2026-07-31T10:23:00"
}
```

memory_extracted 字段存本次对话由 LLM 抽取的记忆要点，供 MemoryController 异步入库，下次对话召回注入提示词。

十、声音克隆模块设计

10.1 模块职责

voiceclone 模块（modules/voiceclone）管理声音克隆任务与声音资源。用户可上传自己的声音样本，系统提交克隆任务至 TTS 服务商（如豆包、GPT-SoVITS），训练完成后生成定制音色，供智能体 TTS 使用。包含 VoiceCloneController（克隆任务）与 VoiceResourceController（声音资源管理）。

10.2 克隆流程

克隆流程：上传声音样本（WAV/MP3）→ 创建克隆任务（记录样本、目标服务商、参数）→ 提交至服务商训练 → 轮询或回调获取训练结果 → 训练成功生成音色 ID → 绑定至智能体 TTS 配置。任务状态（待上传/训练中/成功/失败）由 VoiceCloneService 驱动，失败可重试。样本存于对象存储，元数据落库。

10.3 声音资源

VoiceResourceController 管理声音资源库：预置音色、用户克隆音色、音色试听样本。资源支持分类、标签检索与试听。音色绑定至智能体后，Python 服务端 TTS Provider 按音色 ID 调用对应服务商合成。

10.4 克隆任务状态机

克隆任务状态流转：待上传(UPLOADING) → 训练中(TRAINING) → 成功(SUCCESS)/失败(FAILED)。失败任务保留样本可重试。状态由服务商回调或轮询更新，VoiceCloneService.updateStatus 驱动并落操作日志。训练成功后音色 ID 写回资源库并通知用户。

10.5 接口示例

```
POST /voiceclone/task
body: { "agent_id":"agt-001", "provider":"doubao",
        "sample_url":"oss://voice/sample_xxx.wav" }
resp:  { "code":0, "data":{ "task_id":"vc-001", "state":"UPLOADING" } }

GET /voiceclone/task/{taskId}
```

```
resp: { "code":0, "data":{ "task_id":"vc-001",  
    "state":"SUCCESS", "voice_id":"v-doubao-xxx" } }
```

十一、知识库模块设计

11.1 模块职责

knowledge 模块 (modules/knowledge) 管理知识库与知识文件，为语音对话提供检索增强 (RAG) 能力。运营人员为智能体配置专属知识库，上传文档 (PDF/Word/TXT)，系统解析分块后入向量库，对话时 Python 服务端检索相关知识片段注入提示词。包含 KnowledgeBaseController (知识库 CRUD) 与 KnowledgeFilesController (文件管理)。

11.2 数据模型

KnowledgeBase 记录知识库元数据 (名称、所属智能体、向量集合名)。KnowledgeFiles 记录文件元数据 (文件名、大小、状态、分块数)。文件状态：待解析/解析中/已索引/失败。解析由后台任务异步执行，分块后写入 OceanBase 向量字段或外部向量库。

11.3 检索联动

Python 服务端在对话前根据智能体配置的知识库执行向量检索，召回 Top-K 相关片段拼入提示词。本系统仅管理知识库与文件元数据及触发解析，向量存储与检索由 Python 端的 Memory/知识库 Provider 执行。该分工使管理后台与检索引擎解耦。

11.4 文件解析流程

文件上传后由后台任务异步解析：读取文件 → 文本抽取 (PDF/Word/TXT) → 按固定字符数分块 → 向量化入 OceanBase 向量字段。解析进度由 KnowledgeFiles.status 标记 (待解析/解析中/已索引/失败)。失败任务记录错误日志供重试。该异步设计避免上传接口阻塞。

11.5 知识库接口

```
POST /knowledge/base          // 创建知识库  
GET  /knowledge/base/list     // 知识库列表  
POST /knowledge/files         // 上传知识文件  
GET  /knowledge/files/{id}    // 文件状态查询  
DELETE /knowledge/files/{id}  // 删除文件
```

十二、支付与订阅模块设计

12.1 模块职责

payment 模块（modules/payment）管理订单与支付回调，subscription 模块（modules/subscription）管理订阅周期。两者支撑产品线的付费能力（会员订阅、增值服务购买）。包含 PaymentController（下单、查询）、PaymentNotifyController（支付回调）、MockPaymentNotifyController（测试回调）、SubscriptionController（订阅查询与续期）。

12.2 支付流程

下单：前端调用 PaymentController 创建订单（金额、商品、用户）→ 返回支付参数 → 前端拉起支付 → 支付完成后支付服务商回调 PaymentNotifyController → 校验签名后更新订单状态为已支付 → 触发后续业务（如开通订阅）。回调采用幂等设计（common/aspect 幂等切面），防止重复回调导致重复开通。

12.3 订阅周期

Subscription 管理订阅周期：开始时间、到期时间、状态（有效/过期/取消）。订阅到期由定时任务检查并标记过期，可配置到期前提醒。订阅与智能体或会员权益绑定，过期后权益自动收回。

12.4 订单实体

字段	说明
order_no	订单号，唯一
user_id	用户
product_code	商品编码
amount	金额（分）
state	待支付/已支付/已关闭/已退款
pay_time	支付时间
channel	支付渠道
trade_no	渠道交易号

金额以分为单位整数存储，避免浮点精度问题；trade_no 用于对账。

十三、故事引擎模块设计

13.1 模块职责

storyengine 模块（modules/storyengine，59 个类）管理宠物故事引擎的编排与上下文。故事引擎为蛋宝宝宠物的成长剧情提供分支叙事能力：宠物在不同阶段、不同心情下触发不同故事节点，故事节点可由运营配置分支与结局。包含 StoryEngineController。

13.2 故事节点模型

故事以节点图组织：每个节点含触发条件（阶段/心情/亲密度阈值）、剧情文本、分支选项与下一节点指针。节点支持多分支与回环。运营在智控台可视化编辑节点图，本系统持久化节点关系并供 Python 服务端或小程序查询当前节点与推进。

13.3 上下文推进

PetContext 记录宠物当前所在故事节点。对话或操作触发节点推进判定：StoryEngineService 评估当前上下文是否满足某节点触发条件，满足则推进至新节点并推送剧情。该设计使宠物剧情可由运营动态编排，无需发版。

13.4 故事节点结构

```
{
  "node_id": "n-stage1-morning",
  "trigger": { "stage": "幼体", "mood_min": 60, "time": "morning" },
  "dialogue": "宝宝今天精神不错，要不要一起散步？ ",
  "branches": [
    { "choice": "去散步", "next": "n-walk" },
    { "choice": "再待会", "next": "n-stay" }
  ]
}
```

节点以 JSON 存储，trigger 描述触发条件，branches 描述分支与下一节点指针，支持多分支与回环。

十四、系统管理模块设计

14.1 模块职责

sys 模块（modules/sys，53 个类）提供系统级管理能力：管理员账户、字典类型与字典数据、系统参数、操作日志、服务端管理。包含 AdminController（管理员 CRUD）、SysDictTypeController/SysDictDataController（字典）、SysParamsController（参数）、SysOperationLogController（操作日志）、ServerSideManageController（服务端管理）。

14.2 字典管理

字典（Dictionary）用于管理枚举型业务数据，如设备型号、服务商列表、状态码等。字典类型（DictType）定义分类，字典数据（DictData）定义具体条目（键、值、排序、启用状态）。业务模块通过字典键引用，避免硬编码枚举。字典变更即时生效，前端下拉选项动态加载。

14.3 系统参数

SysParams 管理可配置的系统级参数（键值对），如全局开关、阈值、提示文案。参数支持按键查询与批量更新，变更后缓存至 Redis 供各模块读取。该机制将可调参数从代码抽离至运营可配。

14.4 操作日志

SysOperationLog 记录管理员全部写操作（创建、更新、删除）的执行人、时间、目标对象、前后值。日志通过 common/aspect 的操作日志切面自动采集，无需业务代码显式埋点。日志支持按人、按模块、按时间检索，满足运营审计与合规要求。

14.5 服务端管理

ServerSideManageController 管理服务端实例的元数据（实例标识、地址、状态、负载），供 Python 语音核心服务选择接入实例及负载均衡。该模块支持多实例部署下的服务发现与治理。

14.6 字典数据结构

字段	说明
dict_type	字典类型（如 device_model）
dict_label	显示标签（如 "ESP32-S3"）
dict_value	存储值（如 "s3"）

sort	排序
enable	启用状态
remark	备注

业务模块按 dict_type 拉取全部启用条目填充下拉选项，运营增删字典即时生效。字典数据缓存于 Redis，变更时清缓存重载。

14.7 操作日志切面

操作日志切面基于自定义注解 @Log 标记需记录的 Service 方法。切面环绕方法执行，捕获方法名、模块、参数、返回值、执行人、耗时与异常，异步写入 sys_operation_log 表。该设计将审计能力与业务代码解耦，新增审计点仅需加注解。

```
@Log(title = "智能体配置", action = "update")
public void updateAgent(AgentDTO dto) { ... }
```

十五、数据存储设计

15.1 OceanBase 关系存储

本系统以 OceanBase（MySQL 8.0+ 兼容）作为主关系数据库，存储全部业务数据与配置。MyBatis-Plus 3.5.5 提供通用 CRUD 与条件构造器，DAO 接口继承 BaseMapper 获得开箱即用的增删改查。Druid 连接池管理连接复用与 SQL 监控。表结构按模块划分，命名遵循模块前缀约定（如 pdc_nfc_batch、agent、device）。

15.2 Redis 缓存与会话

Redis 5.0+ 承担两类职责：会话存储（Shiro Token 会话、登录态、权限集合）与业务缓存（字典、系统参数、陪伴上下文、热点配置）。缓存采用 common/redis 工具封装，支持键前缀、TTL、序列化策略。会话 TTL 控制登录有效期，业务缓存 TTL 按数据更新频率配置。

15.3 Liquibase 迁移

数据库变更由 Liquibase 管理，changelog 主文件位于 src/main/resources/db/changelog/db.changelog-master.yaml，按版本聚合子 changelog。每次表结构变更（加字段、加索引、建表）新增 changeset，changeset 含唯一 id 与作者，保证幂等执行。该机制使数据库演进可追溯、可回滚、多环境一致。changelog 结构示例：

```
databaseChangeLog:
  - changeSet:
    id: 202503141335
    author: John
    changes:
      - sqlFile:
        encoding: utf8
        path: classpath:db/changelog/202503141335.sql
  - changeSet:
    id: 202504082211
    author: czc
    changes:
      - sqlFile:
        encoding: utf8
        path: classpath:db/changelog/202504082211.sql
```

规范约定：每次表结构改动只允许新建 changeSet，禁止修改上一个 changeSet，保证迁移历史不可篡改。

15.4 MyBatis-Plus DAO 示例

各模块 DAO 接口继承 BaseMapper<T> 获得开箱即用 CRUD，复杂查询以 XML 或条件构造器声明。DAO 接口与实体示例：

```
// 设备 DAO
@Mapper
public interface DeviceDao extends BaseMapper<DeviceEntity> {
    // 自定义查询：按智能体查绑定设备
    List<DeviceEntity> selectByAgentId(@Param("agentId") String agentId);
}

// 实体
@TableName("device")
public class DeviceEntity {
    @TableId(type = IdType.ASSIGN_ID)
    private String id;
    @TableField("device_code")
    private String deviceCode;
    private String macAddress;
```

```
private String agentId;
private Integer state;
// 通用审计字段
private Long creator;
private Long updater;
private Date createDate;
private Date updateDate;
}
```

条件构造器 LambdaQueryWrapper 支持链式拼接查询条件，减少手写 XML，复杂多表关联仍用 XML 映射。

15.5 对象存储

固件包、声音样本、知识库文件、头像等大文件存于对象存储（OSS），common/oss 提供上传抽象（put/get/签名 URL）。文件元数据（路径、大小、MD5、类型）落库管理，二进制不入库。该分离保证数据库轻量与访问性能。签名 URL 支持限时直链下载，避免服务端转发流量。

十六、接口文档与开放 API 设计

16.1 Knife4j 文档

本系统采用 Knife4j 4.6.0（基于 SpringDoc OpenAPI）自动生成接口文档与调试 UI，访问地址 / xiaozhi/doc.html。文档按模块分组，每个 Controller 的 @Tag 注解定义分组名，@Operation 注解描述接口用途，@Parameter 描述参数。文档自动同步代码变更，无需单独维护，降低文档腐化风险。

16.2 统一响应封装

所有接口返回统一结构 Result<T>，含 code（业务码）、msg（消息）、data（数据载荷）。成功响应 code 为 0 或 200，失败响应携带业务错误码与可读消息。分页接口返回 PageResult，含 records、total、page、limit。该封装使前端可统一处理成功与错误，减少适配成本。

16.3 接口规范

- 路径：统一前缀 /xiaozhi，RESTful 风格，资源名复数
- 方法：GET 查询、POST 创建/动作、PUT 更新、DELETE 删除
- 鉴权：Header 携带 Token，Shiro 过滤器链校验
- 分页：page 与 limit 参数，默认 1/10

- 时间：ISO 8601 或时间戳，统一时区

16.4 响应结构示例

```
// 成功响应
{
  "code": 0,
  "msg": "success",
  "data": { "id": "agt-001", "name": "客厅陪伴助手" }
}

// 分页响应
{
  "code": 0,
  "msg": "success",
  "data": {
    "records": [ {...}, {...} ],
    "total": 128,
    "page": 1,
    "limit": 10
  }
}

// 错误响应
{
  "code": 401,
  "msg": "token 无效或已过期",
  "data": null
}
```

前端拦截器按 code 统一处理：0 放行 data、4xx 跳登录或提示、5xx 提示服务异常并上报。

16.5 接口分组

接口文档按模块分组，每个 Controller 的 @Tag 定义组名（如"设备管理""智能体""NFC 写卡"）。运营与开发可按组快速定位接口，并在线调试。文档随代码 @Operation/@Parameter 注解自动同步，降低文档腐化风险。

十七、配置与部署设计

17.1 配置文件

本系统配置采用 `application.yml`（默认）与 `application-{profile}.yml`（环境覆盖）分层。`application-dev.yml` 配置开发环境的 OceanBase 数据源、Redis、Knife4j、日志级别等。生产环境通过 `application-prod.yml` 或环境变量覆盖敏感配置（数据库密码、密钥）。配置密钥不硬编码，通过环境变量或密钥管理注入。

17.2 打包与运行

Maven 打包生成可执行 jar（`target/xiaozhi-esp32-api.jar`）。运行命令 `java -jar` 启动内嵌 Tomcat，监听 8002 端口。生产部署采用 docker-compose 一键集成，含 `manager-api`、OceanBase、Redis 三个容器，`volumes` 持久化数据与日志。日志输出至 `logs/` 目录，Logback 滚动归档。

17.3 多实例与负载均衡

本系统无状态（会话存 Redis），支持水平扩展多实例。多实例经 Nginx 或负载均衡器分发请求，会话由 Redis 共享保证任一实例均可处理。`ServerSideManageController` 维护实例元数据，供下游 Python 服务端选择接入实例。

17.4 打包与启动命令

```
# 编译打包（跳过测试）
mvn clean package -DskipTests

# 后台启动
mkdir -p logs
nohup java -jar target/xiaozhi-esp32-api.jar \
  --spring.profiles.active=prod \
  > logs/api.log 2>&1 &

# 接口文档地址
# http://localhost:8002/xiaozhi/doc.html
```

启动需 MySQL/OceanBase 与 Redis 就绪，否则 Druid 初始化失败。Spring Boot 启动日志出现 "Started AdminApplication" 即就绪。

17.5 环境隔离

通过 `spring.profiles.active` 区分 `dev/test/prod` 环境，各环境独立 `application-{profile}.yaml`。敏感配置（数据库密码、Token 密钥、支付密钥）经环境变量注入，不入仓库。日志级别按环境配置：`dev` 开 `debug`，`prod` 收敛至 `info` 并落文件。

十八、其他业务模块设计

18.1 companion 陪伴模块

`companion` 模块（`modules/companion`，24 个类）管理陪伴关系与上下文。`CompanionController` 与 `CompanionContextController` 维护用户与宠物的陪伴绑定关系及上下文状态，为宠物养成玩法提供陪伴维度的数据支撑。陪伴上下文与 `PetContext` 协同：前者记录陪伴行为历史（互动次数、最近陪伴时间），后者记录宠物属性状态。

18.2 feedback 反馈模块

`feedback` 模块（`modules/feedback`，10 个类）管理用户意见反馈。`FeedbackController`（用户侧提交）与 `FeedbackAdminController`（管理侧处理）分别服务小程序端提交与智控台处理。反馈含类型、内容、联系方式、状态（待处理/处理中/已回复/已关闭），支持附件上传。该模块为产品迭代收集用户声音。

18.3 invite 邀请模块

`invite` 模块（`modules/invite`，16 个类）管理用户邀请与奖励。`InviteController` 生成邀请码与邀请关系，记录被邀请人注册与奖励发放。邀请关系以树形结构存储，支持多级邀请统计。奖励发放经幂等切面保护，防止重复发放。

18.4 model 模型管理模块

`model` 模块（`modules/model`，16 个类）管理 LLM 模型与模型服务商元数据。`ModelController` 与 `ModelProviderController` 维护可用模型清单（名称、服务商、上下文长度、计价、是否多模态），供智能体配置时选择。模型清单可由运营增删，变更后智能体配置下拉选项即时更新。

18.5 timbre 音色模块

`timbre` 模块（`modules/timbre`，8 个类）管理 TTS 音色库。`TimbreController` 维护预置音色与试听样本元数据，供智能体 TTS 配置选择。音色与 `voiceclone` 模块的克隆音色共同构成音色资源池。

18.6 correctword 纠词模块

correctword 模块 (modules/correctword, 10 个类) 管理 ASR 文本纠词词典。CorrectWordController 维护易错词与纠正映射 (如"小明"→"小名")，配置下发后 Python 服务端 ASR 在产出文本后按词典纠正，提升识别准确率。该模块支持运营按业务术语维护纠词表。

18.7 wechat 微信模块

wechat 模块 (modules/wechat, 17 个类) 管理微信生态对接，含微信登录、支付 (与 payment 协同)、消息推送。WechatController 处理微信 OAuth 登录回调、公众号消息、模板消息推送等。该模块是小程序与微信生态的对接枢纽。

18.8 item 商品与 email/sms 通知

item 模块 (18 个类) 管理商品与虚拟物品；email 与 sms 模块提供邮件与短信通知能力，用于验证码、订阅提醒、订单通知等场景。通知发送统一经 common/service 的通知服务封装，支持模板与限频。

18.9 llm 与 security 辅助

llm 模块 (2 个类) 管理后端侧的 LLM 调用配置 (如智控台内的 AI 辅助功能)；security 模块 (25 个类) 管理登录与会话，与 common 的 Shiro 鉴权协同，LoginController 处理账号密码登录与验证码校验。两者共同构成系统的接入鉴权底座。

十九、关键技术

19.1 分层架构与依赖注入

系统采用 Controller-Service-DAO 三层分层，层间通过 Spring 依赖注入解耦。Controller 仅做协议转换，Service 承载业务，DAO 处理持久化。该架构职责清晰、易测试、易维护，符合 Spring 最佳实践。

19.2 Shiro 轻量鉴权

选用 Apache Shiro 而非 Spring Security，因其配置轻量、过滤器链直观、学习成本低，满足本系统鉴权需求且不引入过度复杂度。过滤器链 + 权限注解的组合支持粗细两级权限控制。

19.3 MyBatis-Plus 通用 CRUD

MyBatis-Plus 的 BaseMapper 提供开箱即用 CRUD，条件构造器（QueryWrapper/LambdaQueryWrapper）支持链式条件拼接，减少手写 XML。分页插件统一分页逻辑。该工具显著降低 DAO 层样板代码。

19.4 Liquibase 版本化迁移

Liquibase 以 changelog 管理数据库变更，changeset 幂等执行，保证多环境（开发/测试/生产）数据库结构一致且可追溯。新增字段无需手动同步各环境。

19.5 操作日志切面

common/aspect 的操作日志切面通过自定义注解 @Log 标记需记录的方法，切面自动采集方法名、参数、返回值、执行人、耗时并异步写入日志表。业务代码无感知，避免显式埋点导致的散落与遗漏。

19.6 配置中枢与热下发

本系统作为产品线配置中枢，将智能体参数集中持久化并供 Python 服务端远程拉取热重载。该设计使 AI 能力运行时可调，无需重启服务，支撑产品线快速迭代与服务商切换。

19.7 幂等切面

支付回调等需幂等的接口通过 common/aspect 幂等切面保护：以业务键为幂等键，首次请求执行并记录，重复请求直接返回首次结果。该机制防止网络重试或重复回调导致的业务重复（如重复开通订阅）。

19.8 模块统计

模块	类数	主要职责
agent	89	智能体配置/快照/声纹/MCP/模板/历史
pdc（nfc）	112	NFC 写卡与领取全链路
sys	53	字典/参数/操作日志/服务端管理
pet	52	宠物档案/记忆/聊天/故事
storyengine	59	故事引擎编排与上下文
device	28	设备注册/绑定/OTA

payment	29	订单/支付回调
knowledge	25	知识库与文件
security	25	登录/鉴权
其余	~	config/voiceclone/model/timbre/wechat 等

全系统共 696 个 Java 类，约 5.7 万行代码，构成完整的智能体管理服务后端。

二十、安全设计

20.1 认证与会话

登录采用账号密码 + 验证码，密码以 BCrypt 散列存储，不存明文。登录成功签发 JWT Token，会话存 Redis 并设 TTL，支持滑动续期与登出失效。Token 密钥配置于环境变量，不硬编码。失败登录限流防爆破。

20.2 鉴权与权限

Shiro 过滤器链校验认证态，@RequiresPermissions 注解校验细粒度权限。权限数据按角色聚合，管理员仅可见其权限范围内的接口与数据。鉴权失败统一返回 401/403，不泄露内部结构。

20.3 输入安全

common/xss 过滤器清除请求参数中的危险脚本，防 XSS。Hibernate Validator 校验参数合法性，防非法输入。MyBatis-Plus 全部使用参数化查询，防 SQL 注入。文件上传校验类型与大小，防恶意文件。

20.4 敏感数据

数据库密码、Token 密钥、支付密钥等敏感配置通过环境变量注入，不硬编码于代码或提交至仓库。日志脱敏，不打印完整 Token 与密码。支付回调校验签名防伪造。该设计满足等保与个人信息保护要求。

20.5 审计

操作日志切面记录全部写操作的执行人、时间、目标、前后值，日志不可篡改，支持按人、模块、时间检索。该审计能力满足业务合规与异常追溯需求。

20.6 安全自检清单

- 密码 BCrypt 散列存储，不存明文
- Token 密钥经环境变量注入，不硬编码
- 数据库密码、支付密钥不入仓库
- 参数化查询，防 SQL 注入
- XSS 过滤器清除危险脚本
- Hibernate Validator 校验全部入参
- 支付回调校验签名防伪造
- 失败登录限流防爆破
- 鉴权失败不泄露内部结构
- 日志脱敏，不打印完整 Token
- 操作日志全量审计写操作